

DEVOPS — LEARNING ROADMAP

v1

20 branches in five themes, from basics to advanced, with the tools for each
foundations · build & deliver · infrastructure · run & operate · secure & scale

THE MAP

work down a theme in order; themes B and C are best learned together, because a pipeline with nothing to deploy onto teaches you half the job

DEVOPS

Learning Roadmap · 20 branches
every branch below is expanded in a card on this sheet

A · FOUNDATIONS

- 1 DevOps fundamentals & culture
- 2 Linux, networking & scripting
- 3 Version control & collaboration
- 4 Cloud platform fundamentals

B · BUILD & DELIVER

- 5 Build & continuous integration
- 6 Testing in the pipeline
- 7 Artifacts, registries & supply chain
- 8 Continuous delivery & GitOps

C · INFRASTRUCTURE

- 9 Infrastructure as Code
- 10 Configuration management
- 11 Containers, images & VMs
- 12 Kubernetes & orchestration

D · RUN & OPERATE

- 13 Deployment & release strategies
- 14 Monitoring & observability
- 15 Reliability, SRE & incident response
- 16 Capacity, performance & FinOps

E · SECURE & SCALE

- 17 DevSecOps & compliance
- 18 Networking & platform services
- 19 Platform engineering & DX
- 20 Database & data operations

HOW TO USE THIS ROADMAP

1. Theme A first, and do not rush it. Almost every DevOps problem that looks hard is a Linux, networking or Git problem underneath.
2. Take B and C together — build a pipeline and something for it to deploy onto, in the same project.
3. Automate one manual thing per week. The skill is noticing the toil, not knowing the tool.
4. Then D, which is where the job actually happens: keeping a running system healthy and knowing when it is not.
5. **Learn one cloud deeply rather than three shallowly. The concepts transfer; the console screenshots do not.**

THE PART THE TOOL LIST CANNOT TEACH YOU

DevOps is an operating model before it is a toolchain. Every tool here exists to shorten the loop between a change being written and that change being safely in production — and to make the result visible enough that you can tell whether it worked.

If a tool does not shorten that loop or widen that visibility, it is overhead you have chosen to carry.

Teams routinely adopt Kubernetes, service meshes and GitOps while still deploying monthly by hand, and conclude DevOps did not work. Measure the loop — deployment frequency, lead time, change failure rate, time to restore — before and after. Those four are the scoreboard.

1 DEVOPS FUNDAMENTALS & CULTURE

WHAT IT IS

- Shared ownership of build, deploy and run — not a team name or a job title
- The lifecycle: plan → code → build → test → release → operate → monitor → back to plan
- CALMS — culture, automation, lean, measurement, sharing
- Agile and Scrum basics; lean principles and flow
- Blameless culture — you cannot get honest incident data from people who expect punishment

THE FOUR DORA METRICS

- Deployment frequency** — how often you release to production
- Lead time for changes** — commit to running in production
- Change failure rate** — share of deployments causing a degradation
- Time to restore service** — how quickly you recover
- First two measure throughput, last two stability. Good teams improve both together — they are not a trade-off

ALSO LEARN

- SU, SLO, SLA and error budgets
- Toil, and why eliminating it is the actual job
- Continuous improvement (Kaizen), value stream mapping

Name the four DORA metrics, not just the acronym. They are the only widely-validated way to say whether a DevOps change helped, and they are what an interviewer means when they ask how you would measure success.

2 LINUX, NETWORKING & SCRIPTING

LINUX — NON-NEGOTIABLE

- Filesystem layout, permissions, ownership, ACLs
- Processes, signals, systemd units, journald
- Package managers, users and groups, sudo
- Disk, inodes, mounts, LVM, memory, swap, OOM killer
- Diagnostics: top htop ss lsof strace dmesg df du

NETWORKING

- TCP/IP subnets and CIDR, routing, NAT
- DNS records and resolution order, TTL, and why deploys wait on it
- HTTP, TLS certificates, chains, expiry
- Firewalls, security groups, ports
- Tools: dig curl ss netstat tcpdump traceroute nc

SCRIPTING

- Bash — pipes, redirection, exit codes, set -euo pipefail, traps
- Python for anything past ~50 lines of Bash
- Text processing: grep sed awk jq yq
- Cron and systemd timers
- Windows and PowerShell for hybrid estates

This branch is the one people skip and then hit a wall on. Nearly every hard production problem — a hung pod, a TLS failure, a full disk, a DNS cache — resolves to Linux or networking, and no amount of tooling substitutes for it.

3 VERSION CONTROL & COLLABORATION

GIT

- Clone, commit, push, pull, fetch; the staging area
- Merge vs rebase vs cherry-pick, and when each is honest
- Conflict resolution, rebase, bisect, stash, tags
- Submodules and monorepo vs polyrepo trade-offs

BRANCHING STRATEGIES

- Trunk-based** — short-lived branches, feature flags. The strategy that correlates with high DORA performance
- GitHub flow** — simple, branch per change
- Git flow** — release branches; heavier, suits scheduled releases and versioned products

COLLABORATION & HYGIENE

- Pull requests, review culture, CODEOWNERS
- Branch protection, required checks, signed commits
- Conventional commits and semantic versioning
- Pre-commit hooks — lint and secret-scan before it ever reaches CI

LONG-LIVED BRANCHES ARE A DELIVERY PROBLEM, NOT A GIT PROBLEM. MERGE PAIN IS A SYMPTOM OF BATCH SIZE; THE FIX IS SMALLER CHANGES BEHIND FLAGS, NOT A CLEVERER BRANCHING MODEL.

4 CLOUD PLATFORM FUNDAMENTALS

Learn one provider properly. The service names differ; the concepts are the same everywhere.

THE CORE PRIMITIVES

- Identity** — IAM users, roles, policies, least privilege, assume-role, OIDC federation from CI
- Networking** — VPC, subnets, route tables, NAT and internet gateways, security groups, peering, private endpoints
- Compute** — VMs, autoscaling groups, managed containers, serverless functions
- Storage** — object, block, file, storage classes and lifecycle rules
- Databases** — managed relational, NoSQL, cache; backups and read replicas
- Regions and availability zones** — what fails together
- Managed load balancers, DNS, CDN, certificates
- Tagging, accounts and landing zones, quotas and limits

Missing from most DevOps roadmaps, and it should not be. IAM, VPC design and autoscaling underpin branches 9 through 18. Learning Kubernetes before cloud networking means debugging both at once.

Use short-lived credentials from CI. OIDC federation between your CI provider and the cloud removes long-lived access keys entirely — the single highest-value cloud security change most teams can make.

5 BUILD & CONTINUOUS INTEGRATION

CI PRINCIPLES

- Everyone merges to trunk at least daily
- Every commit triggers an automated build and test
- A broken build is the team's first priority — stop the line
- Build once, promote the same artifact through environments
- Fast feedback: minutes, not hours. Parallelise and cache

PIPELINE MECHANICS

- Pipeline as code, versioned beside the application
- Stages, jobs, matrix builds, fan-out and fan-in
- Caching dependencies and layers; ephemeral runners
- Reproducible and hermetic builds
- Secrets injection — never in the repo, never in logs
- Self-hosted vs managed runners, and runner isolation

QUALITY GATES

- Linting and formatting, static analysis
- Coverage thresholds that block, not just report
- Build metadata: commit SHA, build number, provenance

Build once, deploy many. Redubling per environment means the thing you tested is not the thing you shipped. Promote an immutable artifact and inject configuration at deploy time.

6 TESTING IN THE PIPELINE

Continuous delivery is only as safe as the tests that gate it. This is the branch that decides whether automation is an accelerator or a faster way to ship defects.

THE PYRAMID

- Unit** — many, fast, isolated. Run on every commit
- Integration** — real dependencies via containers
- Contract** — consumer-driven, so services can deploy independently without an end-to-end suite
- End-to-end** — few, high value, slowest and fakesit
- Smoke** — post-deploy, in the real environment

BEYOND CORRECTNESS

- Performance and load testing with a pass/fail threshold
- Security tests — SAST, DAST, dependency and secret scanning
- Infrastructure tests and policy checks
- Chaos experiments as scheduled tests

PRACTICES

- Test data management and seeding
- Ephemeral environments per pull request
- Quarantine flaky tests immediately — a suite people ignore is worse than no suite

TOOLS
JUnit · pytest · Testcontainers · Pact · Playwright · Cypress · JMeter · Gatling

7 ARTIFACTS, REGISTRIES & SUPPLY CHAIN

ARTIFACT MANAGEMENT

- Immutable, versioned artifacts; never overwrite a tag
- Semantic versioning and release metadata
- Container registries, Helm chart repositories, package registries
- Retention and cleanup — registries grow without bound
- Promotion between repositories rather than rebuilds

SOFTWARE SUPPLY CHAIN

- SBOM** — a bill of materials per build (SPDX, CycloneDX)
- Artifact signing and verification** — Sigstore / cosign
- Provenance and attestation** — what built this, from which source, on which runner
- SLSA** as the maturity framework for build integrity
- Dependency pinning and lockfiles; base image hygiene
- Verify signatures at admission, not just at build

TOOLS

Nexus · JFrog Artifactory · GHCR / ECR / ACR · Harbor · cosign · Syft · Gripe · Dependabot / Renovate

Underweighted in most roadmaps, and now a compliance requirement in many sectors. "Which of our running images contain this CVE?" is unanswerable without SBOMs and a registry you can query.

8 CONTINUOUS DELIVERY & GITOPS

DELIVERY VS DEPLOYMENT

- Continuous delivery** — every change is releasable; the final push may be a decision
- Continuous deployment** — every passing change goes to production automatically
- Progressive delivery — release to a subset first, expand on evidence

PIPELINE STAGES

- Commit → build → test → scan → package → sign → deploy → verify → notify
- Environment promotion with the same artifact
- Automated rollback triggered by health signals, not a human noticing
- Approvals and change records where regulation requires them

GITOPS

- Git is the single source of truth for desired state
- An agent in the cluster continuously reconciles actual to desired
- Drift is detected and corrected automatically
- Rollback is a Git revert; the audit trail is the commit history
- Pull-based, so CI needs no cluster credentials
- Separate application and config repositories

TOOLS

Argo CD · Flux · Spinnaker · Argo Rollouts · Jenkins · GitLab CD

GITOPS IS A MODEL, NOT A PRODUCT. ITS VALUE IS THE RECONCILIATION LOOP AND THE AUDIT TRAIL — DEPLOYING WITH KUBECTL. APPLY FROM CI IS NOT GITOPS JUST BECAUSE THE MANIFESTS LIVE IN GIT.

9 INFRASTRUCTURE AS CODE

CORE IDEAS

- Declare the desired state; let the tool compute the change
- Idempotent — applying twice changes nothing the second time
- Version-controlled and peer-reviewed like application code
- Immutable infrastructure: replace rather than patch in place
- Plan → review — apply, always in that order

PRACTICAL CONCERNS

- State** — remote backend, locking, encryption. Never local, never in Git
- Modules and reusability; a private module registry
- Workspaces and environment separation
- Drift detection** — someone always changes something in the console
- Secrets handling — never in state, never in plan output
- Importing existing resources; blast-radius control by splitting state

TESTING & POLICY

- Static analysis and policy as code before apply
- Integration tests that create and destroy real resources
- Cost estimation on the pull request

TOOLS

Terraform · OpenTofu · Pulumi · CloudFormation · Bicep / ARM · Crossplane · Terragrunt · Terratest · Checkov · tfint · OPA / Conftest · Infracost

10 CONFIGURATION MANAGEMENT

Provisioning creates the machine; configuration management decides what is on it and keeps it that way. Related to branch 9, but a different problem.

CONCEPTS

- Desired-state enforcement and convergence
- Agent-based (Puppet, Chef) vs agentless (Ansible over SSH)
- Push vs pull models
- Inventory, roles, playbooks, idempotent modules
- Templating configuration per environment
- Secrets at config time — vault lookups, not committed files

WHERE IT STILL MATTERS

- VM estates and hybrid or on-premises environments
- Golden image building — bake with Packer, then boot immutable
- Compliance baselines and hardening (CIS benchmarks)
- Patch management across a fleet

TOOLS

Ansible · Chef · Puppet · SaltStack · Packer · AWS SSM · cloud-init

Ansible is configuration management, not infrastructure provisioning. Listing it beside Terraform obscures the distinction: Terraform decides which resources exist, Ansible decides what state they are in. Many teams use both, for different jobs.

11 CONTAINERS, IMAGES & VMs

VIRTUAL MACHINES

- Hypervisors — VMware, KVM, Hyper-V; type 1 vs type 2
- Cloud instance families and sizing; on-prem vs cloud
- Images, templates, snapshots and cloning

- When a VM is still the right answer: licensing, kernel modules, stateful legacy

CONTAINER FUNDAMENTALS

- Namespaces and cgroups — a container is a process, not a small VM
- Images, layers, tags and digests; the OCI standard
- Volumes, bind mounts, networking modes
- Container runtimes: containerd, CRI-O, Docker Engine

IMAGE QUALITY

- Multi-stage builds; minimal or distroless base images
- Layer order for cache efficiency; docker ignore
- Run as non-root**, read-only filesystem, drop capabilities
- Pin base images by digest; rebuild regularly for patches
- One process per container; handle SIGTERM for graceful shutdown
- Scan every image, and again in the registry over time

TOOLS

Docker · Podman · BuildKit · Buildah · Docker Compose · Harbor · Trivy · Dive

12 KUBERNETES & ORCHESTRATION

WORKLOADS & CONFIG

- Pods, ReplicaSets, Deployments, StatefulSets, DaemonSets
- Jobs and CronJobs; init and sidecar containers
- ConfigMaps and Secrets; Namespaces, labels and selectors
- Requests and limits**, QoS classes — the most common source of both waste and eviction
- Liveness, readiness and startup probes
- PodDisruptionBudgets, affinity, taints and tolerations

NETWORKING & STORAGE

- Services, Ingress and the Gateway API; CNi plugins
- NetworkPolicies — default-deny is the goal
- PersistentVolumes, PVCs, StorageClasses, CSI drivers

SCALING & EXTENSION

- HPA, VPA, Cluster Autoscaler, Karpenter
- CRDs and operators; the reconciliation pattern
- Helm and Kustomize for packaging and overlays

RBAC, service accounts, Pod Security Standards

- Admission control and policy (OPA Gatekeeper, Kyverno)

Cluster upgrades, node pools, etcd backup and restore

TOOLS

kubectl · Helm · Kustomize · k9s · Lens · kubectx · kubens · kind · minikube · EKS / AKS / GKE · Karpenter · Kyverno

13 DEPLOYMENT & RELEASE STRATEGIES

STRATEGIES

- Recreate** — stop then start. Downtime, simplest, fine for batch
- Rolling** — replace instances gradually. The default
- Blue/green** — two environments, switch traffic, instant rollback. Doubles cost during the switch
- Canary** — small traffic share first, expand on metrics
- A/B testing** — routing by cohort, for product decisions rather than safety
- Shadow / dark launch** — mirror real traffic to the new version, discard responses

DECOUPLING DEPLOY FROM RELEASE

- Feature flags — ship code dark, release by configuration
- Flag lifecycle and deployment state flags become permanent complexity
- Kill switches for risky paths

MAKING IT SAFE

- Automated rollback on SLO breach, not on someone noticing
- Backward-compatible changes so old and new run side by side
- Health checks and readiness gates before traffic shifts
- Post-deploy smoke tests and bake time

TOOLS

Argo Rollouts · Flagger · LaunchDarkly · Unleash · Spinnaker · service mesh traffic splitting

14 MONITORING & OBSERVABILITY

THE DIFFERENCE

- Monitoring** — watching known failure modes you predicted
- Observability** — being able to answer questions you did not predict, from the telemetry you already emit

THE THREE SIGNALS

- Metrics** — cheap, aggregatable. Watch label cardinality
- Logs** — structured JSON, correlated, sampled at volume
- Traces** — one request across services, with context propagated through async hops
- OpenTelemetry** covers all three, not just tracing — the vendor-neutral instrumentation standard
- Continuous profiling as a fourth signal

KNOWING WHAT TO WATCH

- RED** — rate, errors, duration, for services
- USE** — utilisation, saturation, errors, for resources
- Four golden signals: latency, traffic, errors, saturation
- Dashboards per audience; correlation IDs and to end
- Alert on symptoms and SLO burn rate, not on CPU
- Every alert needs a runbook, or it should not page

TOOLS

Prometheus · Grafana · Loki · ELK / OpenSearch · Jaeger · Tempo · OpenTelemetry · Alertmanager · Datadog · Zabbix

15 RELIABILITY, SRE & INCIDENT RESPONSE

DESIGNING FOR FAILURE

- High availability, redundancy, multi-AZ and multi-region
- Health checks and probes; graceful shutdown and draining
- Timeouts, retries with jitter, circuit breakers, bulkheads
- Graceful degradation and load shedding
- Single points of failure — find them deliberately

CONTINUITY

- RTO** — how long you may be down. **RPO** — how much data you may lose
- Backups, and restores that are **actually rehearsed**
- DR patterns: backup-restore, pilot light, warm standby, active-active
- Failover drills on a schedule

OPERATING

- SLIs and SLOs; error budgets that gate releases
- On-call rotation, escalation, handover
- Incident command: roles, comms, severity levels
- Blameless postmortems** with tracked actions
- Runbooks kept current, and chaos engineering to test them

TOOLS

LinusChaos · Chaos Mesh · Gremlin · Veleo · PagerDuty / Opsgenie · Statuspage · incident.io

A backup you have never restored is a hypothesis. Restore time is a number you measure, not one you assume — and it is usually far worse than expected the first time.

16 CAPACITY, PERFORMANCE & FINOPS

UNDERSTAND THE WORKLOAD FIRST

- Traffic shape: peak vs average, daily and weekly cycles, seasonal events
- Baselines per service — CPU, memory, disk, network, connections
- Growth trend and the headroom you keep
- Load and soak testing to find the actual limit

SCALING

- Vertical vs horizontal; stateless services scale trivially
- Autoscaling on the right signal — queue depth or latency, rarely CPU alone
- Right-sizing requests and limits; scale-to-zero where it fits
- Warm-up and cold-start behaviour

COST

- Tagging and cost allocation — per team, service and environment
- Reserved capacity, savings plans, spot instances
- The usual waste: idle non-production, over-provisioned requests, orphaned volumes and IPs, cross-AZ and egress traffic, log retention
- Cost anomaly alerting; cost visible on the pull request
- Unit cost — cost per request or per tenant, tracked as an engineering metric

TOOLS

k6 · JMeter · Infracost · Kubecost / OpenCost · cloud cost explorers and pricing calculators · Goldilocks · VPA

17 DEVSECOPS & COMPLIANCE

SHIFT LEFT, BUT NOT ONLY LEFT

- Security is everyone's job and belongs at every stage, not a gate at the end
- Threat modelling during design
- Fall the build on high-severity findings — a report nobody blocks on is ignored

SCANNING ACROSS THE PIPELINE

- SAST** — source code · **DAST** — running application
- SCA** — dependencies and licences
- Secret scanning** — pre-commit and in history
- Container and IaC scanning** — images and misconfiguration
- Runtime security** — anomalous behaviour in production
- IDENTITY & SECRETS**
 - Least privilege everywhere; no long-lived cloud keys — federate CI with OIDC
 - Central secrets store, dynamic credentials, automated rotation
 - Kubernetes: External Secrets, sealed secrets, workload identity
 - mTLS between services; network policy default-deny
- COMPLIANCE**
 - ISO 27001, SOC 2, PCI-DSS, GDPR and local data-protection law
 - Audit logging, change records, evidence generation
 - Policy as code so controls are enforced, not documented
 - Separation of duties without blocking delivery

TOOLS

Trivy · Snyk · SonarQube · OWASP ZAP · Semgrep · Checkov · tscs · HashCorp Vault · External Secrets · Falco · OPA · Kyverno · gitleaks

18 NETWORKING & PLATFORM SERVICES

GETTING TRAFFIC IN

- Load balancers — L4 vs L7, health checks, draining
- Ingress controllers and the Gateway API
- DNS management, records, failover routing, TTL strategy
- TLS certificate issuance and automated renewal
- CDN and edge caching; WAF and DDoS protection
- API gateways — rate limiting, quotas, authentication, versioning

TRAFFIC BETWEEN SERVICES

- Service mesh** — mTLS, retries, circuit breaking, traffic splitting, telemetry without touching application code
- Service discovery and health-aware routing
- Sidecar vs sidecarless (eBPF) meshes
- East-west vs north-south — gateways and meshes solve different halves

ASYNCR PLUMBING

- Message queues and event streams as platform services
- Dead letter queues, consumer lag, retention and replay

TOOLS

NGINX · Envoy · Traefik · HAProxy · Istio · Linkerd · Cilium · cert-manager · Route 53 / Cloud DNS · CloudFront / Cloudflare · Kong · Apigee · Kafka · RabbitMQ